Prototyping of A Primary Smart Home System

-BEng (Hons) Electronic Engineering - EL1242 Digital Electronics

Student Name: Lowlife555

Student Number: [redacted]

Date: 2025.3.20

Fall Detection System, Intelligent Lighting Control System and Automatic Heating Control System Based on Mbed Platform

Catalogs

-INTRODUCTION:....3
-Sensors and Actuators:....4
Temperature sensor (DS1621):....4
Light Dependent Resistor (LDR):....4
Three-axis acceleration sensor (ADXL335):....4
Potentiometer....5
-Design Important Points....6
Sensor calibration....6
Low-power design....6
Anti-interference design....6
-Specific mandate requirements....6
Task 1: Automatic heating control....6
Task 2: Automatic Lighting Control....6
Task 3: Fall Detection....6
Task 4: Environmental Parameter Reading....6
-Device pin Assignment:....6
-Task source code....7
-Source code explanation....7
-Read dipswitch status....9
-Analog temperature control system....9
-Demonstration Video....11
-References....11

INTRODUCTION:

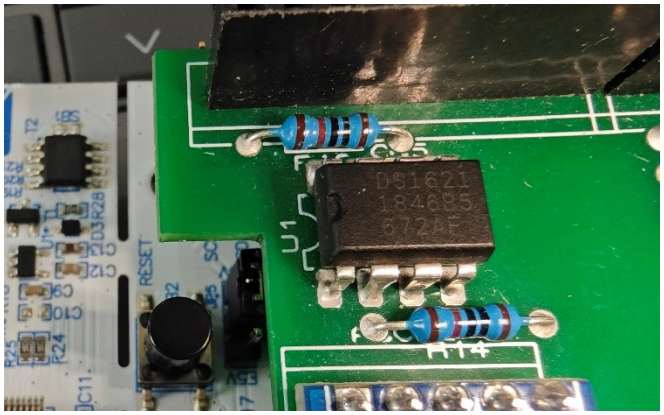
As the global aging trend intensifies, the safety of the elderly is a growing concern. With age, the mobility of the elderly decreases, and according to incomplete statistics, about one-third of the elderly over 65 years old experience one or more falls each year, which greatly affects the quality of life of the elderly and brings great impact on family happiness. The development of microprocessors and embedded systems has fueled the development of smart homes, which have shown great potential in enhancing comfort and quality of life (Nijat E Hasanov, Jun 22, 2023). The aim of this project is to develop an embedded system that accomplishes fall detection, automatic heating control, and smart lighting control functions through accelerometers, temperature sensors, and photoresistors to improve the safety and convenience of life for the elderly.

NUCLEO STM32L476RG development board:

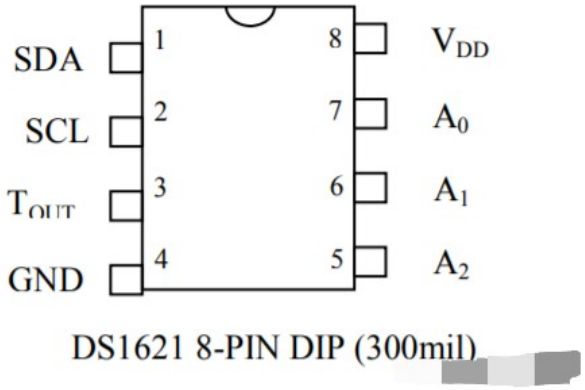
NUCLEO STM32L476RG was born from the innovation of ARM architecture and the deep cooperation with ST (STMicroelectronics). British Acorn self-developed the first generation of ARM chips, due to the use of RISC instruction set, laid the foundation for low-power and high-performance technology, and since then ARM has become an independent company to develop, and gradually become the world's leading semiconductor intellectual property supplier. STMicroelectronics launched the STM32 series of microcontrollers at the beginning of the 21st century, based on the ARM Cortex-M core, which combines the advantages of high performance, low power consumption, and rich peripherals ("ARM Cortex-M4 Architecture-Beginners Guide ", September 23, 2021), the STM32L476RG used this time as the representative model of STM32L4 series, using 90nm process to further optimize the power consumption, and become the core platform in the field of industrial control, IoT, and smart home (June 6, 2024 " STM32 NUCLEO-L476RG Review: A Developer's Best Ally").

Sensors and Actuators:

Temperature sensor (DS1621):



-DS1621 On board

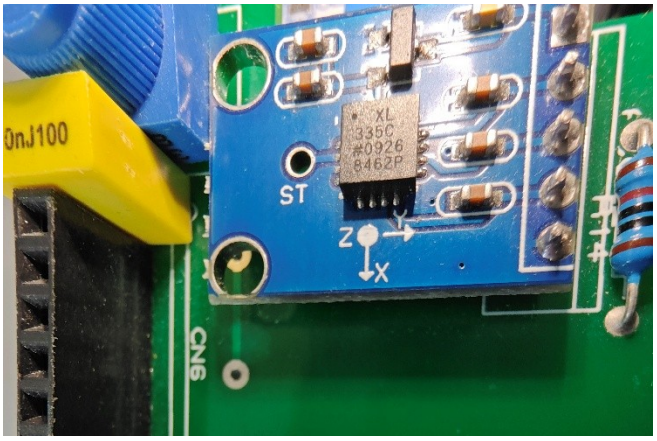


-DS1621 Pinout

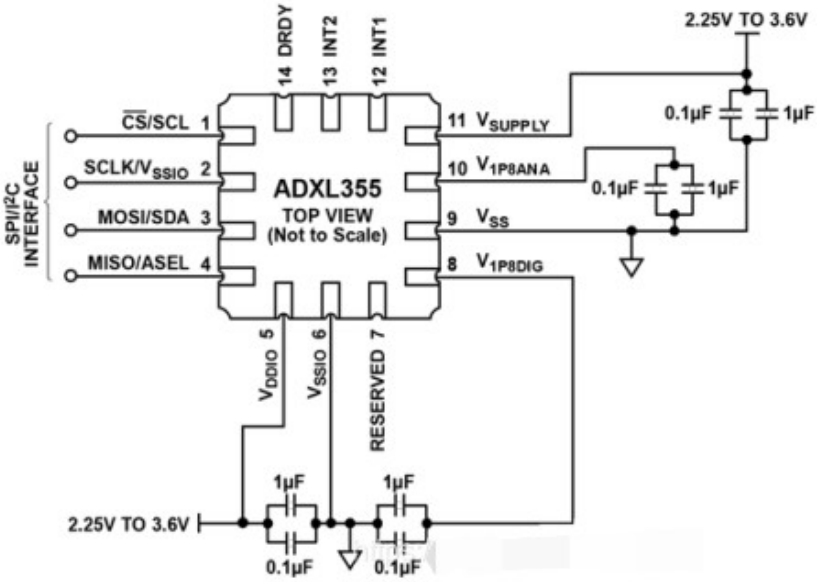
DS1621 is a real-time monitoring of the ambient temperature for automatic heating control and parameter display in task 1.

Light Dependent Resistor (LDR): detects the ambient light intensity and controls the LED lighting (Task 2) and the parameter display (Task 4). It is characterized by an analog output and the voltage value is inversely proportional to the light intensity.

Three-axis acceleration sensor (ADXL335):



-ADXL335 On board



-ADXL335 Pinout

It detects acceleration, can be realized by triggering the fall alarm and parameter display function, the range of the sensor is 3G.

Potentiometer: This module regulates the analog input value, which is used in this task to regulate the parameter display.

4-LED array (D10-D13): for displaying the lighting intensity, 5 levels of brightness from 0-4 LEDs.

Three-color LED: used for cyclic display of red, green and blue in case of a fall alarm

Serial output: display parameters on PC.

Design Important Points:

Sensor calibration: different development boards need to experimentally determine light thresholds and accelerometer benchmarks for LDRs.

Low-power design: The sleep mode of STM32L476RG is realized by using “sleep_for” function to reduce the power consumption during inactivity.

Anti-interference design: analog signals need to be passed through filter circuits to avoid false-touch problems.

Specific mandate requirements:

Task 1: Automatic heating control

Design a thermostat to turn off the heater when the temperature is greater than or equal to 25°C and turn on the heater when it is less than 25°C.

Task 2: Automatic Lighting Control

Adjust the brightness level of the LED lights based on the photoresistor reading.

Task 3: Fall Detection

Detects fall events using a 3-axis accelerometer and triggers an alarm when a fall is detected.

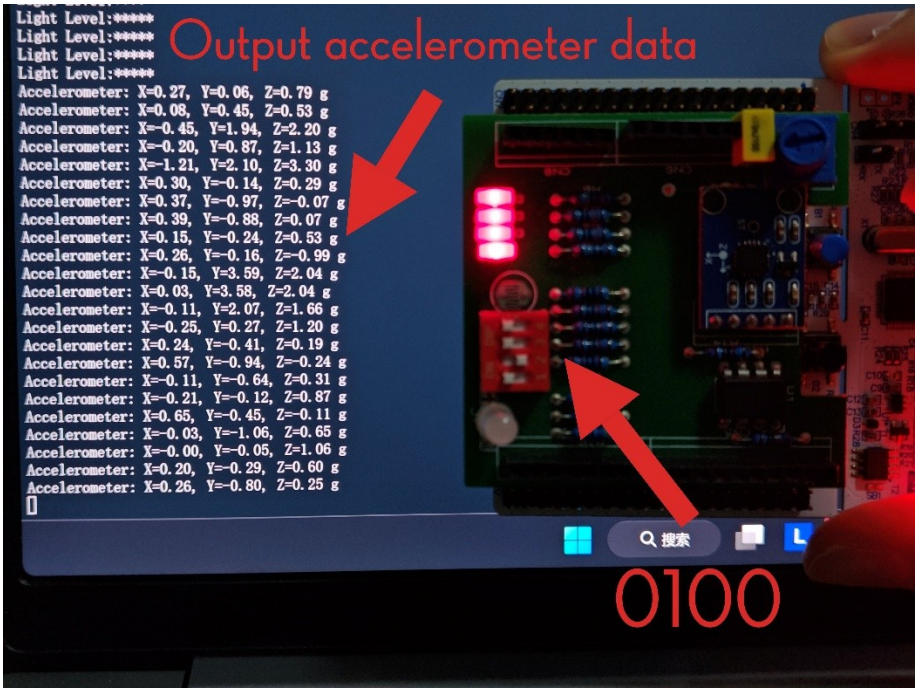
Task 4: Environmental Parameter Reading

Select the parameters to be displayed (temperature, light intensity, acceleration value, potentiometer voltage) by a combination of keystrokes and switches, and send the selected parameters to the PC screen for display via serial communication

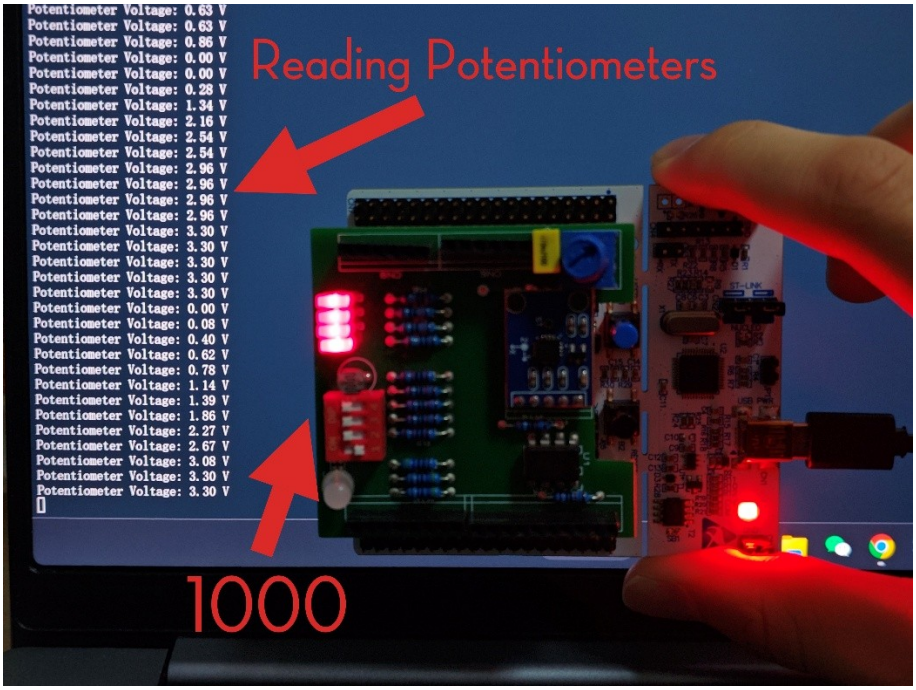
Device pin assignment:

Device	Nucleo/Arduino Pin(s)	ARM Pins	Comment
4 LEDs	D10–D13	PB6, PA7, 6, 5	Can output 4 bits, 1 ‘hex’ digit Nucleo Board ‘LED1’ is D13
4 switches	D6-D9	PB10, PA8, PA9, PC7	Can input 4 bits, one ‘hex’ digit
Push Switch	USER_BUTTON	PC13	Interrupt, Polling
PWM	Any digital output pin, except D6, ARM PB10		Brightness of LEDs, speeds of motors, angular position of

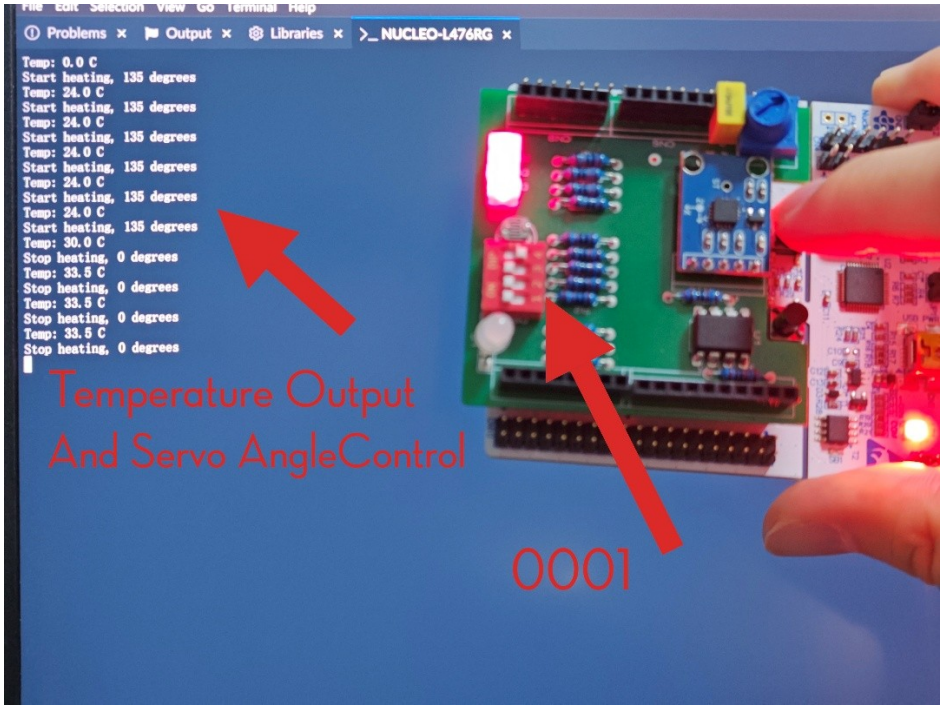
	[we use this for input on our system]		Servos can all be controller with PWM
Multi-coloured LED	D2 = Red D3 = Blue D4 = Green	PA10 PB3 PB5	3 colour LED, allowing (almost) any colour to be produced by addition
Potentiometer	A4	PC1	Adjustable Analogue input
Light dep. Resistor	A5	PC0	Voltage Vin varies inversely with light level
Analogue Accelerometer	A0 = X A1 = Y A2 = Z	PA0 PA1 PA4	ADX335 3 axis analogue accelerometer Axes are marked on module
I^2C sensor	D14 = SDA D15 = SCL	PB9 PB8	DS1621 temperature sensor



-Output accelerometer data



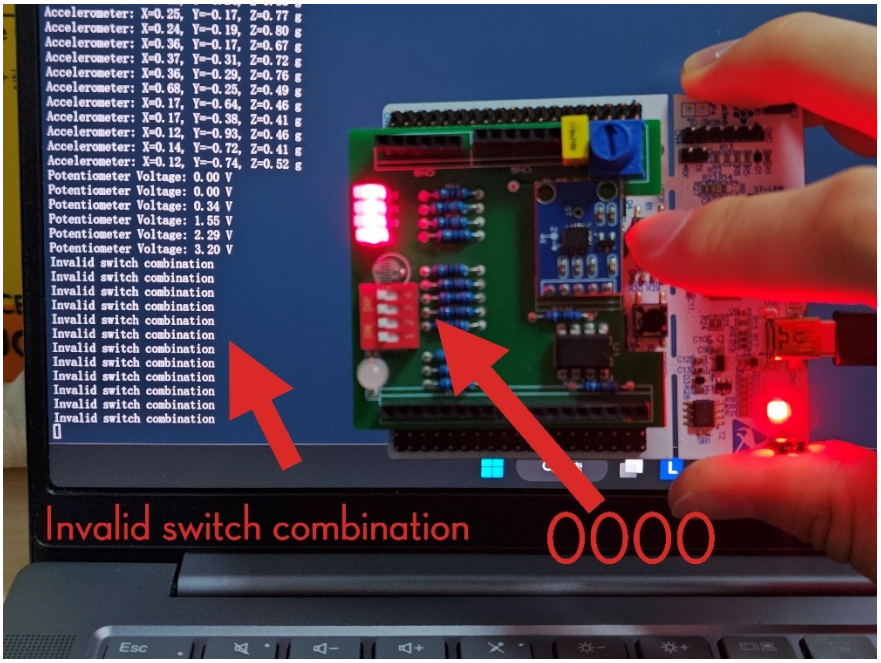
-Reading Potentiometer's data



-Output Temperature Date and Servo Angle degree



-Output ambient light intensity



-Invalid switch combination

Task source code:

Youjia_Full.cpp mbed_app.json

Source code explanation:

```
// Accelerometer
AnalogIn accel_x(PA_0);       // X-axis (A0)
AnalogIn accel_y(PA_1);       // Y-axis (A1)
AnalogIn accel_z(PA_4);       // Z-axis (A2)
// BUTTON1
DigitalIn user_button(BUTTON1);

// Toggle pin (D6-D9 corresponds to PB10, PA8, PA9, PC7)
DigitalIn sw1(PB_10);
DigitalIn sw2(PA_8);
DigitalIn sw3(PA_9);
DigitalIn sw4(PC_7);


// LED (D10-D13 homologous PB6, PA7, PA6, PA5)
DigitalOut led1(PB_6);
DigitalOut led2(PA_7);
DigitalOut led3(PA_6);
DigitalOut led4(PA_5);

// Servo
//PwmOut servo(PA_0);

// Use the BufferedSerial
BufferedSerial pc(USBTX, USBRX);

// Potentiometer (A4)
AnalogIn potentiometer(PC_1);


// Photoresistors (A5)
AnalogIn ldr(PC_0);

// 3-color LEDs
DigitalOut red_led(PA_10);     // Red (D2)
DigitalOut blue_led(PB_3);     // Blue (D3)
DigitalOut green_led(PB_5);    // Green (D4)
int pre_button=1;
```

-Pin Definitions

```
// Servo Angle Control Function
void set_servo_angle(float angle) {
    float duty = (angle / 180.0f) * 0.15f + 0.05f;
    //servo.write(duty);
}
```

-Servo Angle Control Function

```
// Synthetic triaxial acceleration to determine falls
bool detect_fall() {
    float x = accel_x.read() * 3.3f;
    float y = accel_y.read() * 3.3f;
    float z = accel_z.read() * 3.3f;
    float g_x = (x - 1.65f) / 0.330f;
    float g_y = (y - 1.65f) / 0.330f;
    float g_z = (z - 1.65f) / 0.330f-1;
    float magnitude = g_x * g_x + g_y * g_y + g_z * g_z;
    return (magnitude > 3.2);
}
```

-Synthetic triaxial acceleration to determine falls

```
173     //Read dialing status
174     int switch_val = (sw4.read() << 3) | (sw3.read() << 2) | (sw2.read() << 1) | sw1.read();
175     switch_val &=0x0f;
```

-Read dipswitch status

Read dipswitch status:

***sw1 ~ sw4** represent 4-digit dip switches, which form a 4-digit binary number:

***sw4** is in the highest bit (<< 3)

***sw3** in the second highest bit (<< 2)

***sw2** in the second lowest bit (<< 1)

***sw1** in the lowest bit

Combine to form “switch_val” (0~15).

“switch_val &= 0x0f”; ensure data is valid (clear high bit interference).

```
177         case 8: { // Read the temperature
178             float temp = temp_sensor.read();
179             sprintf(buf, "Temp: %.1f C\n", temp);
180             pc.write(buf, strlen(buf));
181             //Analog Servo Control
182             if(temp<25) pc.write("Start heating, 135 degrees\n", 28);
183             else pc.write("Stop heating, 0 degrees\n",25);
184             break;
185         }
```

-Analog temperature control system

Analog temperature control system:

*Reads the **DS1621** temperature sensor’s data and outputs the temperature value (Temp: xx.x C) through the serial port.

*If the temperature is < 25°C, output “Start heating, 135 degrees”.

*Otherwise, output “Stop heating, 0 degrees”.

```
186         case 4: {  // Output light intensity
187             pc.write("Light Level:", 13);
188             for (int i = 5; i > light_level; --i) {
189                 char star[] = "*";
190                 pc.write(star, 1);
191             }
192             //float k = ldr.read();
193             //sprintf(buf,"light level num:%.2f\n",k);
194             //pc.write(buf, strlen(buf));
195 
196             pc.write("\n", 1);
197             break;
198         }
```

-Output light intensity

Output “Light Level:” and then the “*” symbol to indicate the light intensity:

Light Level 4 → “****”

Light Level 3 → “***”

Light Level 2 → “**”

Light Level 1 → “*”

Light Level 0 → “”

```
199     case 2: {  // Output triaxial acceleration
200         float x = (accel_x.read() * 3.3f - 1.65f) / 0.330f;
201         float y = (accel_y.read() * 3.3f - 1.65f) / 0.330f;
202         float z = (accel_z.read() * 3.3f - 1.65f) / 0.330f;
203         sprintf(buf,"Accelerometer: X=%.2f, Y=%.2f, Z=%.2f g\n", x, y, z);
204         pc.write(buf, strlen(buf));
205         break;
206     }
```

-Output accelerometer data

```
207     case 1: {  // Read potentiometer voltage
208         float voltage = potentiometer.read() * 3.3f;
209         sprintf(buf,"Potentiometer Voltage: %.2f V\n", voltage);
210         pc.write(buf, strlen(buf));
211         break;
212     }
213     default:
214         pc.write("Invalid switch combination\n", 28);
215 }
```

-Reading Potentiometers

Demonstration Video：

- *Full-featured demo video*

References:

1:Nijat E Hasanov,Jun 22, 2023 ***"Intro to Intelligent Lighting Control: Enhancing Efficiency and Sustainability in Buildings."***

https://medium.com/@nijat.e.hasanov/intro-to-intelligent-lighting-control-enhancing-efficiency-and-sustainability-in-buildings-6590690f2b99

(Accessed at Mar 17, 2025)

2: September 23, 2021 ***"ARM Cortex-M4 Architecture–Beginners Guide"***

https://microdigisoft.com/arm-cortex-m4-architecture/#google_vignette

(Accessed at Mar 17,2025)

3: June 6, 2024 ***"STM32 NUCLEO-L476RG Review: A Developer's Best Ally"***

https://electronikz.com/products/stm32-nucleo-l476rg-review

(Accessed at Mar 17,2025)